IBT COLLEGE • CODEOPS PROJECT

BUILD AN ONLINE STORE

A step-by-step student roadmap

Plan it → Build it → Test it → Improve it → Present it

Your mission

Create a polished, responsive online store for a clear audience. Customers should be able to browse products, view details, add products to a cart, update quantities, and submit a simple order request. The project also demonstrates the Next.js performance and environment concepts from CodeOps Day 43.

How to use this roadmap

Follow the phases in order. Do not start by coding every page at once. At each phase, first understand what the customer needs, then build the smallest working version, test it, and only then move forward.

Important scope: This project does not include payment processing. Do not build payment screens, request card details, or store payment credentials. The order flow ends with an order confirmation.

Project outcome

By the end, your team should have…	Evidence
A working shopping journey	Home → catalogue → product detail → cart → order form → confirmation
Readable, reusable code	Components, clear naming, and a sensible project structure
A responsive interface	The core journey works on mobile, tablet, and desktop
Performance improvements	Image/font/script choices and a short before/after measurement report
Safe configuration	Environment example file; no secrets committed to Git

Phase 0 — Decide what you are building

Before writing code, agree on a small, realistic store idea. Choose products that are easy to explain and display. Your instructor must approve the idea.

Step	What to do	Why we need it
1. Choose the audience	Write one sentence: “Our store helps ___ find/buy ___.”	A clear audience helps you choose the right products, language, and design.
2. Choose a store type	Examples: books, electronics, fashion, Ethiopian crafts, coffee, home decor, beauty products.	A focused catalogue is easier to finish and test.
3. Choose 8–12 products	Prepare each product’s ID, name, price, category, image, short description, and stock status.	The catalogue, product details, filters, and cart all need consistent data.
4. Sketch the journey	Draw the screens and arrows between them before coding.	You can see missing pages and confusing steps early.
5. Split the work	Agree on page/component ownership, shared data format, and Git workflow.	Teams can work in parallel without duplicating or breaking each other’s work.

Do this before coding

Create a simple wireframe on paper or in a design tool. It does not need to be beautiful. It needs to show what appears on each screen and where each button takes the customer.

Suggested screen map

Screen	What it must show
/ — Home	Store name, navigation, featured products, and a clear “Browse products” action.
/products — Catalogue	Product cards with image, name, price, category, and simple filter or sort.
/products/[id] — Product details	Larger image, description, price, availability, and Add to cart.
/cart — Shopping cart	Chosen products, quantities, remove action, subtotal, and empty-cart state.
/checkout — Order form	Customer name, contact, delivery address, validation, and order summary.
/order-success — Confirmation	Confirmation message and summary/reference for the submitted order.

Phase 1 — Set up the project

Goal: get a clean Next.js application running before building the store.

Do this:

- ■ Create or open the class Next.js project.
- ■ Run it locally and confirm the starter page loads.
- ■ Create a Git repository and make the first commit.
- ■ Remove starter content you do not need.
- ■ Create folders for components, data, and shared utilities as appropriate.
- ■ Add a README with the project name, purpose, and how to run it.

Why? If the app cannot run at the beginning, you will not know whether later errors come from setup or from your feature code.

Checkpoint: Every team member can run the project locally and understands how to start it.

Phase 2 — Create the product data

Goal: make one reliable source for the products, so the same information can be reused on the home page, catalogue, detail page, and cart.

Do this:

- ■ Create a product data file (for example, `src/data/products.js`).
- ■ Give every product a unique ID.
- ■ Use the same field names for every product: id, name, price, category, image, description, and available.
- ■ Add 8–12 realistic sample products.
- ■ Check that all image paths and prices are valid.
- ■ Create a reusable ProductCard component using the shared data.

Why? If every page has its own copy of the product details, prices and names can become inconsistent. Shared data keeps the store easier to update.

Checkpoint: You can display a product card from your data file without copying the product details into the component.

Phase 3 — Build the catalogue and product detail

Goal: let customers discover an item and understand what they are choosing.

Do this:

- ■ Build the home page with a small featured-products section.
- ■ Build the catalogue grid from the shared product data.
- ■ Add at least one useful category filter or sort option.
- ■ Make each product card link to its detail page.
- ■ Build the product detail page using the product ID from the route.
- ■ Add a clear Add to cart button, even if cart logic is implemented in the next phase.
- ■ Add a useful not-found state for an invalid product ID.

Why? Customers need a clear path from discovering a product to understanding its details. Links and page states make that path predictable.

Checkpoint: A customer can browse, filter, open a product, and return to the catalogue.

Phase 4 — Make the cart work

Goal: keep track of the items the customer has selected and calculate totals correctly.

Do this:

- Choose one cart state approach taught in class (for example, React state or the agreed state library).
- Add a product to the cart from a product card/detail page.
- If the same product is added again, increase its quantity rather than creating confusing duplicates.
- Show the selected items, unit price, quantity, and line total.
- Let customers increase/decrease quantity and remove an item.
- Calculate subtotal from the product data and quantities; do not hard-code the total.
- Show a helpful empty-cart message and a link back to products.
- Test quantities of 1, 2, and 0/removal.

Why? Cart behavior is the core interactive part of the project. Correct calculations and clear feedback matter more than fancy animation.

Checkpoint: Add, update, remove, and subtotal actions all work after several different interactions.

Phase 5 — Submit an order (no payments)

Goal: let a customer submit their delivery details and receive a clear confirmation.

Do this:

- Build a checkout form with customer name, contact information, and delivery address.
- Label every field and show which fields are required.
- Validate missing or invalid fields and display understandable messages.
- Show an order summary before submission.
- Prevent submission when the cart is empty.
- When valid, create a simple order object using the cart and customer details.
- Show the order confirmation page with a summary or generated reference.
- Clear the cart after successful submission, if that matches the app design.

Why? The customer needs to know what they ordered and where it will be delivered. Validation prevents incomplete submissions.

> **Keep the scope focused**
>
> There is no payment step in this project. Do not ask for card numbers, bank details, payment PINs, or payment passwords. A successful order form submission should lead directly to the confirmation page.

Checkpoint: A customer can submit valid delivery details and see a confirmation. Empty carts and invalid forms are handled clearly.

Phase 6 — Make the interface responsive and accessible

Do this:

- Test the store at mobile, tablet, and desktop widths.
- Make the navigation and cart usable on a small screen.
- Use semantic headings, buttons, links, and form labels.
- Add descriptive alt text to meaningful product images.
- Make focus visible when navigating with a keyboard.

- Check contrast, text size, spacing, and button size.
- Add empty, loading, invalid, and error states where appropriate.

Why? The project should work for different devices and users, not only on the developer’s laptop.

Checkpoint: Another student can complete the shopping journey on a phone-sized screen without your help.

Phase 7 — Apply CodeOps Day 43 concepts

Do not leave optimisation until the last minute. First make the feature work, then measure, improve, and record the result.

Concept	What to do in this project	Why we need it
next/image	Use Next.js Image for product and hero images where appropriate; set dimensions or use a suitable fill layout.	Helps the browser handle images efficiently and reduces layout shifts.
Remote image hosts	If images come from an external host, configure only the required host in next.config.	The app needs an explicit, controlled list of allowed remote image sources.
next/font	Use the font-loading approach taught in class rather than relying on a third-party font request at runtime.	Improves consistency and can reduce external font-loading work.
next/script	Only load a third-party script if the project truly needs it; choose an intentional loading strategy.	Unnecessary scripts can slow down the page and add JavaScript work.
Reduce JavaScript	Keep mostly static product/content UI in Server Components when suitable; use Client Components only for interactions needing state, effects, or event handlers.	Less client-side JavaScript can improve responsiveness and simplify the app.
Core Web Vitals	Measure LCP, CLS, and INP using the tools demonstrated in class; note the page and conditions tested.	These metrics help explain loading, visual stability, and interaction experience.
Environment files	Use local environment configuration only if needed; commit a safe .env.example, not actual secrets.	Separating configuration from source code makes setup clearer and safer.
Public/private variables	Remember that NEXT_PUBLIC_ values are exposed to browser code. Never put private secrets there.	Anything bundled into the browser must be treated as public.
Environments	Understand how the app differs in development, preview, and production; test a production build.	A project can behave differently outside local development.
next.config	Keep only the configuration needed for the project and explain any non-default settings.	A small, intentional configuration is easier to review and maintain.

Measure honestly

Record your starting measurement, make a specific improvement, then measure again under similar conditions. Do not invent performance scores or claim a metric improved if your test does not show it.

Phase 8 — Test the whole customer journey

Test	Expected result
Open home page	Navigation and featured products are visible.
Browse and filter	The catalogue updates or sorts as expected.
Open product detail	Correct product information appears.
Add two different items	Both appear in the cart with correct prices.
Change a quantity	Line total and subtotal recalculate correctly.
Remove an item	The item disappears and subtotal updates.

Test	Expected result
Open an empty cart	A helpful message and a way back to products appear.
Submit an invalid form	The form explains what needs to be corrected.
Submit a valid order	The confirmation page shows the correct summary.
Use a small screen	No important controls are clipped or unusable.
Build for production	The production build completes successfully.

Phase 9 — Document, demonstrate, and submit

A finished project is more than code that works on one machine. Another developer should be able to understand it, run it, and see what you improved.

Required deliverables

- **Source code:** complete Next.js project in the agreed Git repository.
- **README.md:** project purpose, features, setup instructions, and run commands.
- **.env.example:** variable names and safe placeholder values only, if environment variables are used.
- **PERF.md:** what you measured, which page/device/tool you used, results before and after, and what you changed.
- **Screenshots:** home, catalogue, product detail, cart, checkout form, confirmation, and a mobile view.
- **Demo:** a short live walkthrough of the complete customer journey.

Suggested team timeline

Milestone	Target outcome
Milestone 1	Store idea, audience, wireframe, product data, app runs locally.
Milestone 2	Home page, catalogue, reusable product cards, product detail.
Milestone 3	Working cart: add, quantity, remove, subtotal, empty state.
Milestone 4	Validated order form and confirmation page; no payment features.
Milestone 5	Responsive/accessibility pass, image/font/script improvements, measurements.
Milestone 6	Test, fix bugs, write README and PERF.md, rehearse demo.

Suggested project structure

Use the conventions already taught in class if they differ. This is one possible starting point:

```
src/
app/
page.jsx # home
products/page.jsx # catalogue
products/[id]/page.jsx # product detail
cart/page.jsx # cart
checkout/page.jsx # order form
order-success/page.jsx # confirmation
components/
ProductCard.jsx
ProductGrid.jsx
AddToCartButton.jsx
CartSummary.jsx
data/
products.js
lib/ # shared helpers, if needed
```

Final checklist — are you ready to submit?

Tick every item your team has tested. If something is not finished, say so honestly and explain what remains.

- ■ The app starts and the production build succeeds.
- ■ Product information is consistent across pages.
- ■ Catalogue filtering or sorting works.
- ■ Product detail pages show the correct item.
- ■ Add, quantity, remove, and subtotal work correctly.
- ■ Empty cart and invalid form states are handled.
- ■ Valid order submission reaches confirmation.
- ■ There are no payment pages or payment credential fields.
- ■ The site works on mobile and desktop.
- ■ Product images have appropriate alt text and layout dimensions.
- ■ Client-side JavaScript is used only where needed.
- ■ LCP, CLS, and INP are discussed in PERF.md with honest test notes.
- ■ No secrets or real credentials are committed.
- ■ README, .env.example (if relevant), and PERF.md are included.
- ■ Every team member can explain the part they worked on.

Assessment guide

Area	Marks	What strong work looks like
Shopping journey and functionality	30	Pages connect correctly; cart totals and form validation work; confirmation is clear.
UI, responsive design, accessibility	20	Consistent design, usable at different widths, semantic elements and labels.
Code structure and reuse	15	Shared product data, reusable components, readable naming, sensible component boundaries.
Performance and Day 43 concepts	20	Appropriate image/font/script choices, considered client JavaScript, honest Core Web Vitals measurements.
Configuration and documentation	10	Clear README, safe environment example, useful PERF.md, no committed secrets.
Demo and explanation	5	Team can explain decisions, test the flow, and answer questions.

Questions you should be ready to answer

- • Why did you put product data in one shared file?
- • Why should a product card usually not need to be a Client Component?
- • Which part of your store needs client-side state, and why?
- • How did you prevent the subtotal from becoming incorrect?
- • What do LCP, CLS, and INP tell us?
- • What did you change after measuring performance?
- • Why should .env files with real secrets not be committed?
- • Which environment variables are visible in the browser?

The main rule

Build one small working part at a time. After each phase, test it and show it to a teammate. A complete, understandable store is better than many unfinished features.

Good luck — plan clearly, build steadily, test carefully, and be ready to explain your decisions.